UNIVERSITY of WEST FLORIDA

Machine Learning with Scikit-learn

Module 100: Clustering

Brian Jalaian, Ph.D.
Associate Professor
Intelligent Systems & Robotics Department

Table of Contents

Unsupervised Machine Learning

Unsupervised Machine Learning Review

Advantages

- Discover hidden structures in data (we do not know the right answer upfront)
- Find something interesting in unlabeled data

Supervised vs unsupervised learning

- **Supervised learning:** learn from data labeled with the "right answers"
  - Data comes with inputs x and output y
- **Unsupervised learning:** find something interesting in unlabeled data
  - Data only comes with inputs x, but not output y

Supervised learning

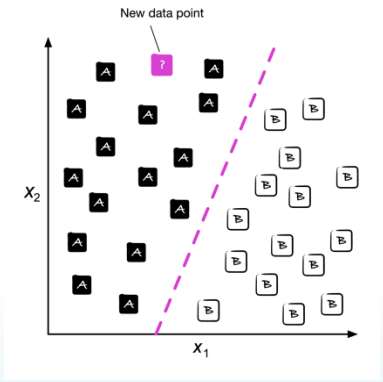


Figure 1.3:
Classifying a new data point

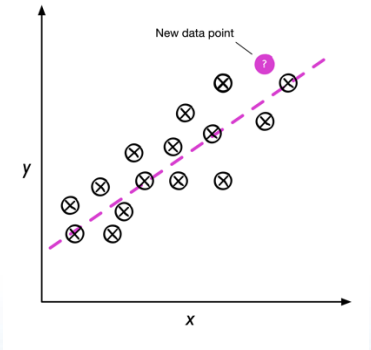


Figure 1.4:
A linear regression example

Unsupervised learning

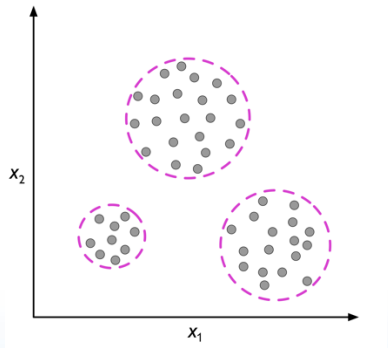


Figure 1.6:
How clustering works

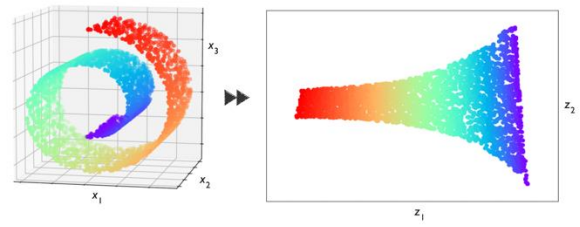


Figure 1.7:
Dimensionality Reduction (3D to 2D)

3 Common Types of Unsupervised Learning

UNIVERSITY of WEST FLORIDA

Types	Clustering	Dimensionality Reduction	Anomaly Detection
Description	group similar data points together	compress data using fewer numbers	find unusual data points
Use Case	market segmentation, anomaly detection, document categorization	data visualization, data compression, feature extraction, noise reduction	fraud detection, network security, fault detection, system health monitoring
Common Algorithms	K-means, hierarchical clustering, DBSCAN	PCA, autoencoders	isolation forest, DBSCAN, autoencoders

Clustering Overview

Description: group similar data points together

Use Case: market segmentation, anomaly detection, document categorization

The 3 common clustering approaches:

1. **Prototype-based clustering:** *K-means*
2. **Connectivity-based clustering:** *Agglomerative Hierarchical Clustering*
3. **Density-based clustering:** *DBSCAN*

Figure 10.5:
K-means clustering

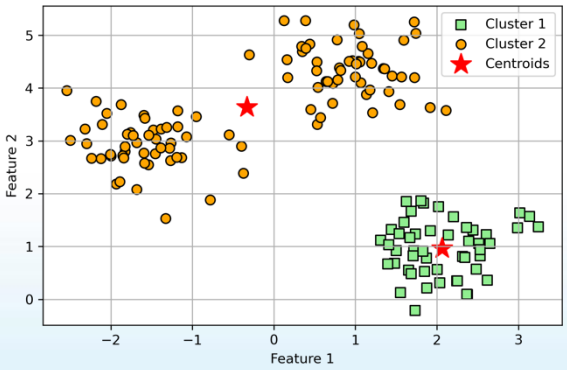


Figure 10.7:
Agglomerative Complete Linkage

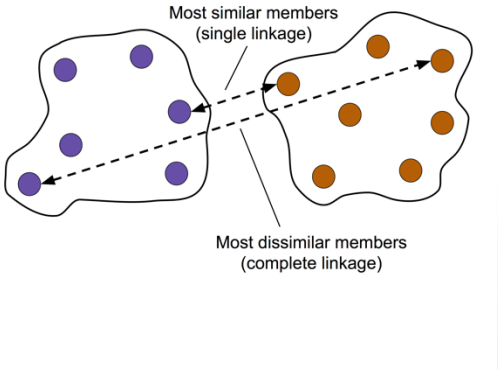
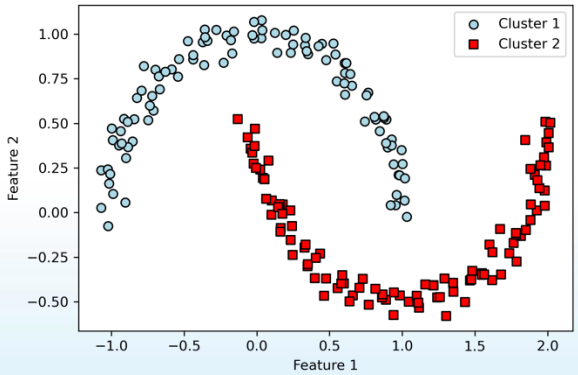


Figure 10.16:
DBSCAN on the half-moon-shaped dataset



K-means Clustering

K-means Clustering

Overview

- It groups the examples* based on their *feature similarities*
  - Define *similarity = opposite of distance of examples*

Drawbacks

- It has to specify the number of clusters as a priori
- Otherwise, bad clustering or slow convergence

Solution to drawbacks

- *Elbow method*
- *Silhouette plots*

→ To evaluate quality of a clustering and help determine optimal number of clusters

Hard vs soft clustering

- **Hard clustering:** assign an example to exact one clusters
  - Ex: *k-means, k-means++ algorithm*
- **Soft clustering:** assign an example to one or more clusters
  - soft clustering = fuzzy clustering
  - Ex: *Fuzzy C-means (FCM) algorithm*

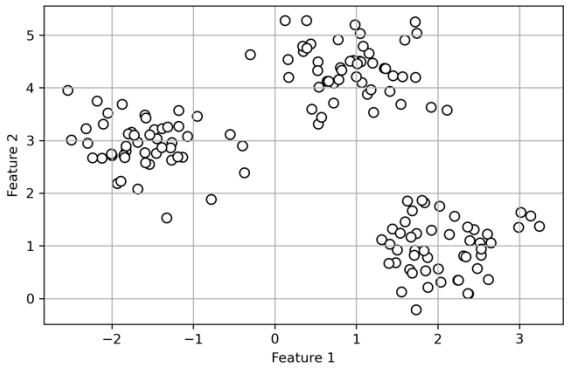
*example = feature = data point = x

K-means Algorithm

Example for intuition

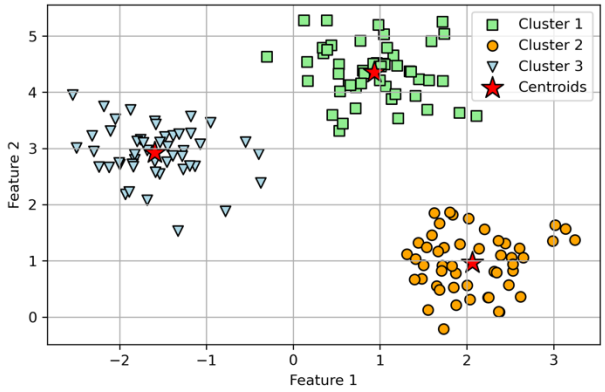
1. Randomly assign cluster centroids, then assign each point to its closest centroid.
2. Recompute centroids, then do it again.

Figure 10.1:
A scatterplot of our unlabeled dataset



until converge

Figure 10.2:
K-means clusters and centroids



Algorithm

1. Randomly pick *k* centroids from the examples as initial cluster centers
2. Repeat until the cluster assignments do not change, or a user-defined tolerance, or maximum number of iterations is reached { $\mu^{(j)}, j \in \{1, \ldots, k\}$ }
   - Assign each example to the nearest centroid,
   - Move the centroids to the center of the examples that were assigned to it

K-means Algorithm

UNIVERSITY of WEST FLORIDA

Math behind the K-means algorithm

- Recall that algorithm groups the examples based on their *feature similarities*
- When *similarity* increase, *distance of examples* decrease
  - Define *similarity = opposite of distance of examples*
- Commonly used distance is *squared Euclidean distance*
  - Distance between two points *x* and *y*, in *m*-dimensional space:

$$d(x,y)^2 = \sum_{j=1}^{m}\left(x_j - y_j\right)^2 = |x-y|_2^2$$

- x, y = example inputs
- m = dimension
- j = j^{th} dimension of the example inputs
  = cluster index

Within-cluster SSE (= Cluster inertia)

- Based on Euclidean distance metric → describe k-means algorithm as a simple optimization problem → minimize *within-cluster sum of squared errors (SSE)*
- *Within-cluster SSE = cluster inertia*
  - Inertia can be interpreted as a measure of distortion
  - Lower value of inertia (lesser distortion) → better clustering

$$SSE = \sum_{i=1}^{n}\sum_{j=1}^{k} w^{(i,j)} \left|x^{(i)} - \mu^{(j)}\right|_2^2$$

- i = index of the example (data record)
- $\mu^{(j)}$ = representative point (centroid) for cluster j
- $w^{(i,j)}$ = membership indicator =

$$w^{(i,j)} = \begin{cases} 1, & \text{if } x^{(i)} \in j, \text{ (example } x \text{ in cluster } j) \\ 0, & \text{otherwise} \end{cases}$$

K-means++ Algorithm

UNIVERSITY of WEST FLORIDA

Overview

- It improves clustering results through more clever seeding of initial cluster centers.

Advantages

- **Faster Convergence**
  - Properly initialized centers can lead to faster convergence of the k-means algorithm.
- **Better Solutions**
  - Reduces the risk of getting stuck in a sub-optimal solution due to poor initialization.

Algorithm

1. Initialize an empty set, M, to store the k centroids being selected.
2. Randomly choose the first centroid, $\mu^{(j)}$, from the input examples and assign it to M.
3. For each example, $x^{(i)}$, that is not in M, find the minimum squared distance, $d\left(x^{(i)}, M\right)^2$, to any of the centroids in M.
4. To randomly select the next centroid, $\mu^{(p)}$, use a weighted probability distribution equal to $\frac{d\left(\mu^{(p)}, M\right)^2}{\sum_i \left(x^{(i)}, M\right)^2}$. For instance, we collect all points in an array and choose a weighted random sampling, such that the larger the squared distance, the more likely a point gets chosen as the centroid.
5. Repeat steps 3 and 4 until k centroids are chosen.
6. Proceed with the classic k-means algorithm.

FCM Algorithm

UNIVERSITY of WEST FLORIDA

Fuzzy C-means algorithm

- Also known as **soft k-means algorithm**, or **fuzzy k-means algorithm**
  - Soft clustering = fuzzy clustering
  - Assign an example to one or more clusters (with probabilities)

Notations

- In k-means, express *cluster membership of an example x* with a sparse vector of binary values:

$$\begin{bmatrix} x \in \mu^{(1)} & \rightarrow w^{(i,j)} = 0 \\ x \in \mu^{(2)} & \rightarrow w^{(i,j)} = 0.9 \\ x \in \mu^{(3)} & \rightarrow w^{(i,j)} = 0.1 \end{bmatrix}$$

- assume *number of clusters* $k = 3$, $j \in \{1, 2, 3\}$
- *example x* is assigned to the cluster centroids μ
- $w^{(i,j)}$ = *membership indicator vector = cluster membership probability*
  - all $w^{(i,j)}$ sum to 1

Algorithm

1. Specify the number of k centroids and randomly assign the cluster memberships for each point
2. Compute the cluster centroids, $\mu^{(j)}, j \in \{1, \ldots, k\}$
3. Update the cluster memberships for each point
4. Repeat steps 2 and 3 until the membership coefficients do not change or a user-defined tolerance or maximum number of iterations is reached

FCM Algorithm

Cost function of FCM

$$J_m = \sum_{i=1}^{n} \sum_{j=1}^{k} w^{(i,j)^m} |x^{(i)} - \mu^{(j)}|_2^2$$

- J_m = *cost function of FCM*
- m = *fuzziness coefficient = fuzzifier*
  - any number greater than or equal to one (typically $m=2$)
  - control degree of fuzziness
  - larger $m \rightarrow$ smaller cluster membership $w \rightarrow$ fuzzier cluster

Cluster membership probability $w^{(i,j)}$

- Calculate the membership of $x^{(i)}$ belonging to the $\mu^{(j)}$ cluster as follows:

$$w^{(i,j)} = \left[\sum_{c=1}^{k} \left(\frac{|x^{(i)} - \mu^{(j)}|_2}{|x^{(i)} - \mu^{(c)}|_2} \right)^{\frac{2}{m-1}} \right]^{-1}$$

- The center $\mu^{(j)}$ of a cluster itself is calculated as the mean of all examples weighted by the degree to which each example belongs to that cluster $w^{(i,j)m}$:

$$\mu^{(j)} = \frac{\sum_{i=1}^{n} w^{(i,j)^m} x^{(i)}}{\sum_{i=1}^{n} w^{(i,j)^m}}$$

Evaluate Clustering: Elbow Method

Overview

Based on the within-cluster SSE (cluster inertia*), we can use a graphical tool to estimate the optimal number of clusters *k* for a given task

- *k* decrease → inertia (distortion) increase
- The idea is to identify the value of *k* where the inertia (distortion) begins to increase most rapidly

Example

- The "elbow" is located at k = 3, so this is supporting evidence that k = 3 is indeed a good choice for this dataset

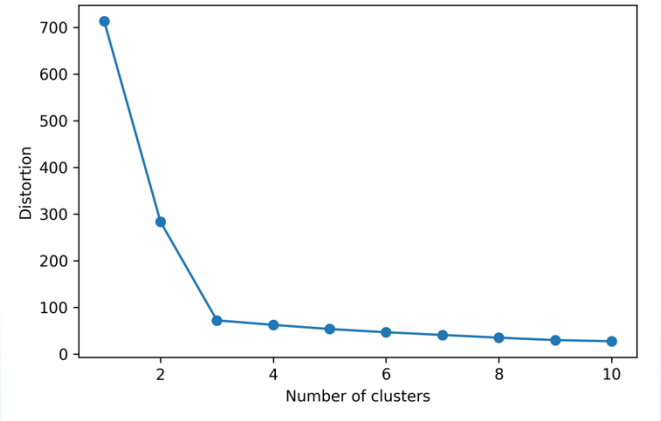


Figure 10.3: Elbow method

* recall that inertia is a measure of distortion

Evaluate Clustering: Silhouette Plots

Overview

- It plots a measure of how tightly grouped the examples in the clusters are
  - Typically we don't have the luxury of visualizing datasets in two-dimensional scatterplots in real-world problems (data in higher dimensions).
  - A measure of how similar an object is to its own cluster (*cohesion*) compared to other clusters (*separation*)
- Help in model selection and parameter tuning

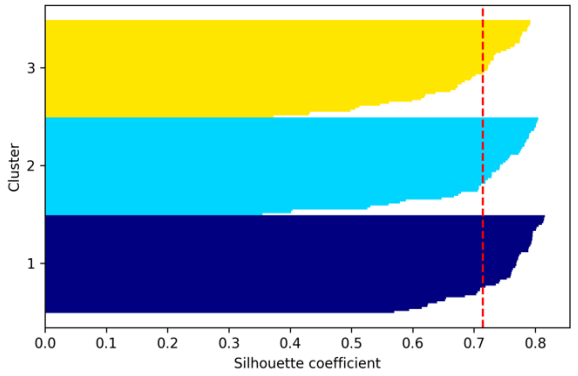
Usage

- Commonly used to determine the best number of clusters
- By computing the *silhouette coefficient (s)* for various numbers of clusters, you can choose the number that gives the highest silhouette coefficient

Limitations

- Computationally intensive for large datasets
- Provides just one perspective on cluster quality

Figure 10.4: Silhouette plot



Evaluate Clustering: Silhouette Plots

UNIVERSITY of WEST FLORIDA

Calculate silhouette coefficient in 3 steps

1. Calculate the *cluster cohesion (a)* as the average distance between an example and all other points in the same cluster
2. Calculate the *cluster separation (b)* from the next closest cluster as the average distance between the example and all examples in the nearest cluster
3. Calculate the *silhouette coefficient (s)* as the difference between cluster cohesion and separation divided by the greater of the two:

$$s^{(i)} = \frac{b^{(i)} - a^{(i)}}{\max\{b^{(i)}, a^{(i)}\}}$$

Properties of silhouette coefficient

- s bounded in the range [-1, 1]
- $s = 0$, if $b = a$
- $s = 1$, if $b >> a$
  - since b quantifies how dissimilar an example is from other clusters,
  - and a tells us how similar it is to the other examples in its own cluster

Evaluate Clustering: Silhouette Plots

Good silhouette coefficients

1. Silhouette coefficients are not close to 0.
2. Silhouette coefficients are approximately equally far away from the average silhouette coefficients (dotted line in the figure).

Example

- 3 clusters is better than 2 clusters, shown in the following figures

Figure 10.4:
Good silhouette coefficients (3 clusters)

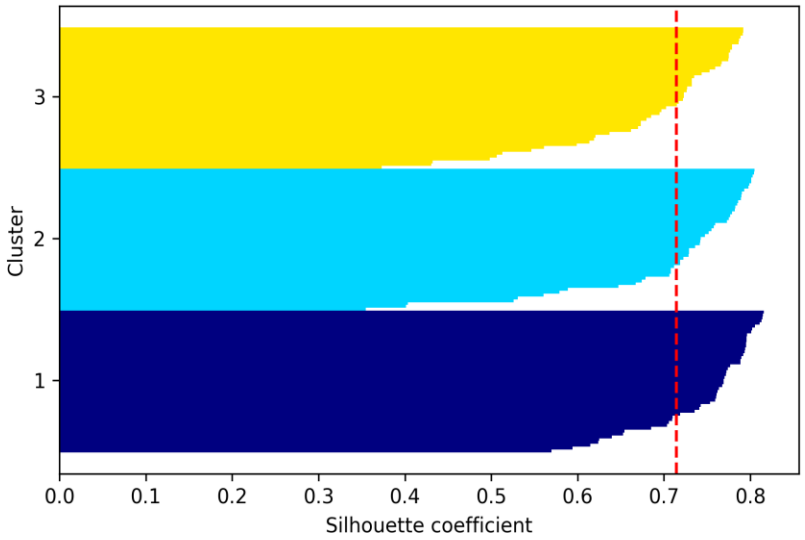
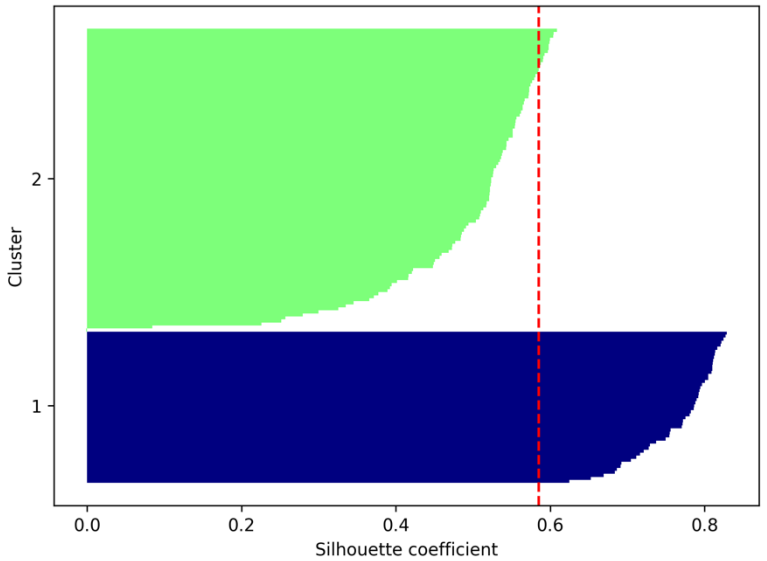


Figure 10.6:
Bad silhouette coefficients (2 clusters)



Hierarchical Clustering

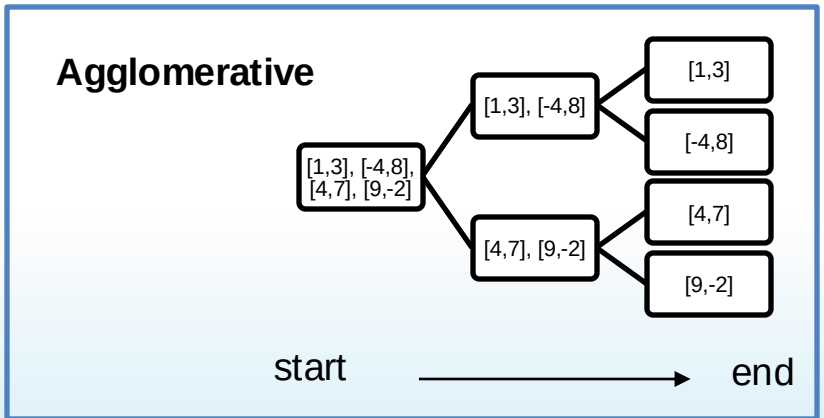
Hierarchical Clustering

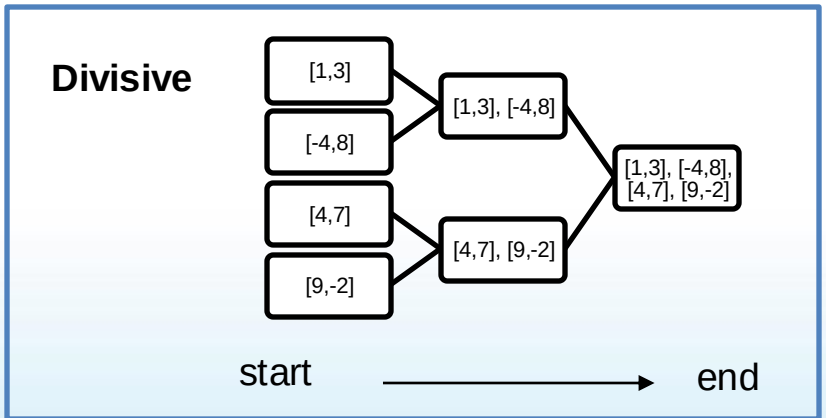
Advantages

- Allows us to plot dendrograms (meaningful taxonomies)
- Not need to specify the number of cluster upfront

The 2 main approaches

1. **Agglomerative hierarchical clustering** (bottom-up approach)
   - start with each example as an individual cluster and merge the closest pairs of clusters until only one cluster remains
1. **Divisive hierarchical clustering** (top-down approach)
   - start with one cluster that encompasses the complete dataset, and we iteratively split the cluster into smaller clusters until each cluster only contains one example





Agglomerative Complete Linkage (ACL)

The 2 standard agglomerative algorithms

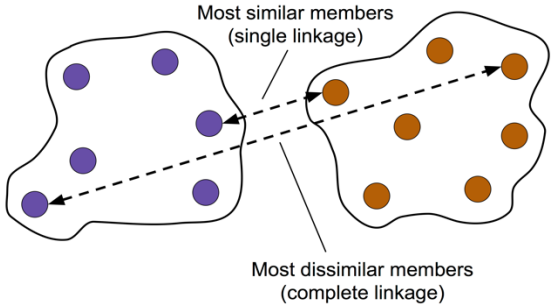
1. **Single linkage**

- Compute the distances between the "most similar" members for each pair of clusters and merge the two clusters for which the distance between the most similar members is the smallest

1. **Complete linkage**

- Similar to single linkage but, compare the "most dissimilar" members in each pair of clusters to perform the merge

Figure 10.7:
Agglomerative Complete Linkage



Agglomerative Complete Linkage (ACL)

ACL algorithm

1. Compute a pairwise distance matrix of all examples
   - calculate Euclidean distance between each pair of **input examples (ID)** in our dataset based on the **features (X, Y, Z)** → pairwise distance matrix
1. Represent each data point as a singleton cluster
2. Merge the two closest clusters based on the distance between the most dissimilar (distant) members
3. Update the cluster linkage matrix
4. Repeat steps 2-4 until one single cluster remains

Example

Original dataset

	X	Y	Z
ID_0	6.964692	2.861393	2.268515
ID_1	5.513148	7.194690	4.231065
ID_2	9.807642	6.848297	4.809319
ID_3	3.921175	3.431780	7.290497
ID_4	4.385722	0.596779	3.980443

→

Euclidean Distance

$$\sqrt{\sum_{i=1}^{n} (x_{i+1} - x_i)^2}$$

→

Pairwise distance matrix

	ID_0	ID_1	ID_2	ID_3	ID_4
ID_0	0.000000	4.973534	5.516653	5.899885	3.835396
ID_1	4.973534	0.000000	4.347073	5.104311	6.698233
ID_2	5.516653	4.347073	0.000000	7.244262	8.316594
ID_3	5.899885	5.104311	7.244262	0.000000	4.382864
ID_4	3.835396	6.698233	8.316594	4.382864	0.000000

Agglomerative Complete Linkage (ACL)

ACL algorithm

1. Compute a pairwise distance matrix of all examples
2. Represent each data point as a singleton cluster
3. Merge the two closest clusters based on the distance between the most dissimilar (distant) members
4. Update the cluster linkage matrix
5. Repeat steps 2-4 until one single cluster remains

Example

Pairwise distance matrix

	ID_0	ID_1	ID_2	ID_3	ID_4
ID_0	0.000000	4.973534	5.516653	5.899885	3.835396
ID_1	4.973534	0.000000	4.347073	5.104311	6.698233
ID_2	5.516653	4.347073	0.000000	7.244262	8.316594
ID_3	5.899885	5.104311	7.244262	0.000000	4.382864
ID_4	3.835396	6.698233	8.316594	4.382864	0.000000

put closest (= min distance) in same cluster, ex: (ID_0, ID_4)
then recalculate distance between other clusters
the most distant to ID_3 in (ID_0, ID_4) is ID_0 → 5.899885

Linkage matrix

	row label 1	row label 2	distance	no. of items in clust.
cluster 1	0.0	4.0	3.835396	2.0
cluster 2	1.0	2.0	4.347073	2.0
cluster 3	3.0	5.0	5.899885	3.0
cluster 4	6.0	7.0	8.316594	5.0

1st & 2nd col: most dissimilar members in each cluster
3rd col: distance between those members
4th col: count of the members in each

Agglomerative Complete Linkage (ACL)

ACL algorithm

1. Compute a pairwise distance matrix of all examples
2. Represent each data point as a singleton cluster
3. Merge the two closest clusters based on the distance between the most dissimilar (distant) members
4. Update the cluster linkage matrix
5. Repeat steps 2-4 until one single cluster remains

Example

Linkage matrix

	row label 1	row label 2	distance	no. of items in clust.
cluster 1	0.0	4.0	3.835396	2.0
cluster 2	1.0	2.0	4.347073	2.0
cluster 3	3.0	5.0	5.899885	3.0
cluster 4	6.0	7.0	8.316594	5.0

1st & 2nd col: most dissimilar members in each cluster
3rd col: distance between those members
4th col: count of the members in each cluster

visualize

Dendrogram

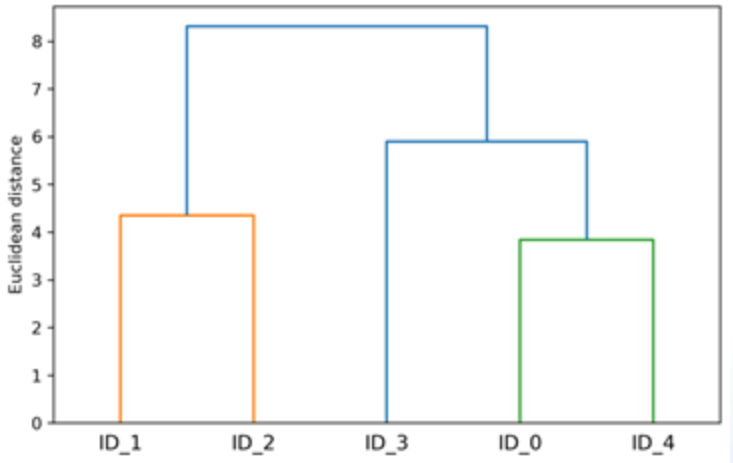


Figure 10.11: A dendrogram of our data

Heat Map for Dendrogram

Advantages

1. **Insightful visualization:** Combining the heatmap with dendrogram allows researchers to visually inspect both the original data values and the clustering structure simultaneously
2. **Patterns and outliers:** The combined visualization can help in identifying patterns, clusters, and potential outliers in the data
3. **Comparing observations:** By clustering both rows and columns, one can compare both individual data points and different observations/experiments

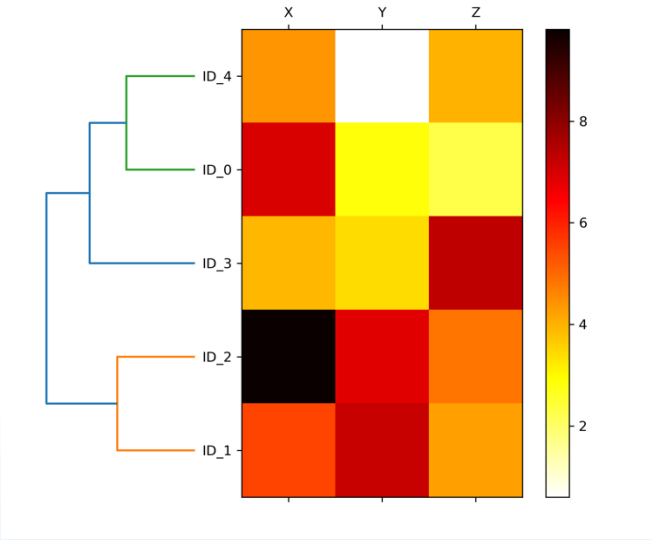


Figure 10.12: A heat map and dendrogram of our data

UNIVERSITY of WEST FLORIDA

DBSCAN

DBSCAN

Overview

- *Density-based Spatial Clustering of Applications with Noise (DBSCAN)*
- Density-based clustering assigns cluster labels based on dense regions of points
  - density is defined as the number of points within a specified radius ϵ

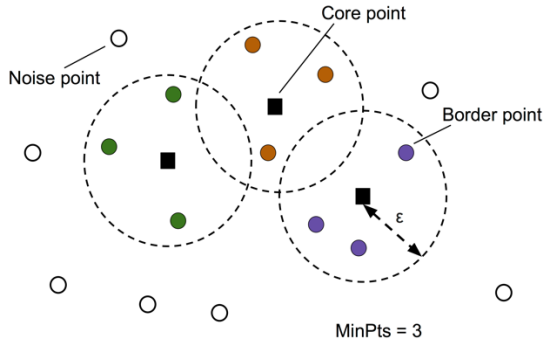


Figure 10.13:
Core, noise, and border points for DBSCAN

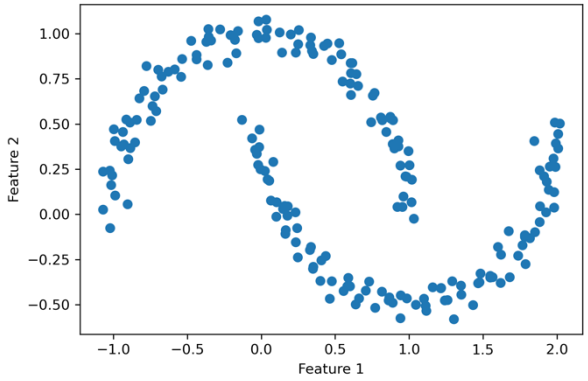


Figure 10.14:
A two-feature half-moon-shaped dataset

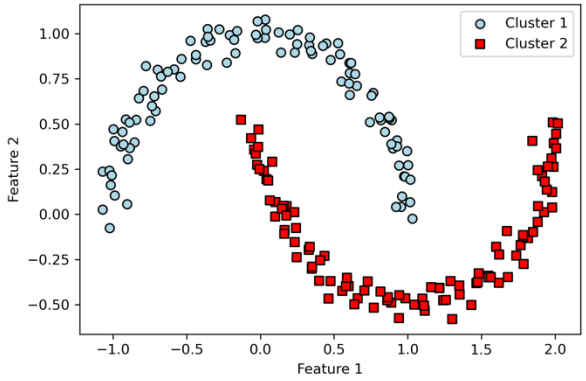


Figure 10.16:
DBSCAN clustering on
the half-moon-shaped dataset

DBSCAN

Labeling data points

- *Core point*: if at least a specified number (MinPts*) of neighboring points fall within specified radius ε
- *Border point*: has fewer neighbors than MinPts within ε, but lies within the ε radius of a core point
- *Noise point*: other points that are neither core nor border points are considered

DBSCAN algorithm

1. Form a separate cluster for each core point or connected group of core points (Core points are connected if they are no farther away than ε)
2. Assign each border point to the cluster of its corresponding core point

*MinPts = number of data points in a neighborhood to define a core point

DBSCAN

UNIVERSITY of WEST FLORIDA

Advantages

1. **No Need to Specify Number of Clusters:** Unlike k-means, where you need to specify *k* (the number of clusters), DBSCAN determines the number of clusters based on the data and parameters
2. **Arbitrary Shapes:** DBSCAN can find clusters of varying shapes, making it suitable for datasets where clusters aren't spherical
3. **Handles Noise:** DBSCAN is good at separating noise from clusters

Disadvantages

1. **Parameter Sensitivity:** The results can vary significantly based on the chosen values of ε and MinPts
2. **Difficulty with Clusters of Varying Densities:** DBSCAN may struggle if clusters in the data have significantly different densities
3. **High Dimensional Data:** Like many clustering algorithms, DBSCAN can become less effective as dimensionality increases